Bewerbungsaufgabe - 3 verliert

Autor: Nico Rautert (Thopia: https://github.com/thopia03)

Wichtig: Hiermit erkläre ich, dass ich diese Aufgabe **ohne den Einsatz von generativer KI** löse. Sollte es doch zum Einsatz von dieser kommen, um z.B. eine grobe Komplexitätsanalyse oder Inspiration für mögliche Frameworks zu bekommen, so wird dies **explizit gekennzeichnet** und der Kontext des Chats mit Link angegeben zum Überprüfen & Nachlesen.

Der Code zu der Aufgabe befindet sich im Repository https://github.com/Thopia03/3VerliertGame

Ebenfalls ist mir im Nachhinein aufgefallen, dass GitHub-Copilot wohl aktiviert war. Ich habe Copilot nicht aktiv genutzt, um mir Code generieren zu lassen.

Organisation und Einführung

Bevor ich mich inhaltlich in die gegebene Aufgabe einarbeite, möchte ich ein paar organisatorische Dinge klären.

Dieses Dokument wurde mit Jupyter-Notebook geschrieben, da sich dies nach meiner Erfahrung für solche Aufgaben, bei welchen man seine Gedankengänge dokumentiert, sehr gut anbietet.

Um meinen Arbeitsprozess nachvollziehbarer zu machen, erkläre ich im Folgenden meine Gliederung. Diese dient für mich zur Orientierung im Entwicklungsprozess als auch zum Verständnis.

1) Anforderungsanalyse

Orientiert an meinem Wissen, beinhaltet die Anforderungsanalyse folgende Punkte:

=> funktionale Anforderungen (Funktionen die das Programm bedienen soll)
=> nicht-funktionale Anforderungen (Dienstqualität, Sicherheit, Umsetzbarkeit, etc.)
=> weitere Anforderungsarten (z.B. Architekturanforderungen)
=> Randbedingungen (Programmiersprachen, Frameworks, Kontextumgebung der zu bauenden Software, etc.)

==> Spezifikation durch Priorisierung & Akzeptanzkriterien

Oftmals werden hier verschiedene Praktiken eingesetzt, um eine vernünftige Anforderungsanalyse zu gewährleisten, wie Interviews, Nutzerbeobachtung, Brain Storming, Use Cases, User Stories, Prototypen, etc. Wichtig ist auch die Klärung des Arbeitsframeworks je nach Gegebenheiten (V-Modell, SCRUM, ...).

2) Problemzerlegung / Algorithmen

In diesem Schritt schauen wir uns an, welche Probleme vorliegen, und zerkleinern diese, wenn möglich, in einfachere und besser verständliche Probleme. Ebenfalls werden sich Gedanken darüber gemacht, wie wir unsere gegebenen Probleme algorithmisch beschreiben können. Dies hilft später enorm bei der Implementierung.

=> Bsp.: Wir haben ein Spiel gegeben, welches ein Hexgrid besitzt. Auf diesem werden verschiedene Teilchen platziert, wobei wir verschiedene Muster haben. Diese Muster haben laut Regelwerk verschiedene Arten und Weisen, um Punkte zu erzeugen.

===> Eine berechtigte Frage hier wäre, wie wir die Berechnung der Punkte algorithmisch modellieren können bzw. welche Ansätze wir nutzen würden. Wenn es z.B. darum geht, eine maximale Kette an Zeichen zu berechnen, dann können wir unser Feld hier als einen Graphen modellieren. Knoten wären die Felder und die Kanten die jeweiligen Nachbarschaften. Wenn wir jetzt die maximale Kette berechnen wollen, bietet sich ein Algorithmus wie Tiefensuche oder Breitensuche an.

===> Diese Idee müsste natürlich dann etwas besser durchdacht und dokumentiert werden, allerdings reicht dies hier aus, um meinen Punkt zu verdeutlichen.

3) Modellierung / Verständnis

Auf Basis unserer Anforderungsanalyse fangen wir nun an, erste erhobene Anforderungen zu modellieren, um ein technisches Verständnis zu bekommen.

Hier können verschiedene Arten von Diagrammen zum Einsatz kommen.
=> Use-Case-Diagramme, Aktivitätsdiagramme, etc.

Resultierend auf Basis dieser Diagramme ist es uns möglich, die Abläufe der einzelnen Schritte zu verstehen.

4) Architektur und Strukturdesign

Natürlich müssen wir uns ebenfalls Gedanken um unsere Software-Architektur machen. Dazu gehört nicht nur klassischerweise die Modellierung unserer Klassen die wir später implementieren (UML-Klassendiagramme), sondern auch die Modellierung der Verantwortlichkeiten unserer Komponenten, deren Schnittstellen und der einzelnen Schichten selbst.

Auch hier gibt es verschiedene Diagramme & Arbeitsweisen.
=> UML-Komponentendiagramme, UML-Verteilungsdiagramme, etc.
=> ADD 3.0 könnte eingesetzt werden, um iterativ unsere Architektur zu designen.

5) Implementierung

Die Implementierung ist natürlich das Herzstück unserer Software. Hier werden alle Anforderungen nach gegebener Architektur implementiert.

6) Testing

Das Testing verläuft meist parallel zur Implementierung, um die Funktionalität zu gewährleisten.

Typisch sind Tests wie:
=> Unit-Tests, Integrationtests, End-to-End-Tests, etc.

7) Bugs / Fehlerbehandlung / Anforderungschecking

Üblich beim iterativen Arbeiten überprüfen wir in diesem Schritt, ob unser Programm Fehler beinhaltet.

=> Bugs: führen meist zu einer Re-Iteration der Implementierung & des Testings
=> falsche / fehlende Anforderungen: Dieser Schritt ist meist deutlich kostenintensiver, da wir unsere Entwicklung von vorn an neu anpassen müssen.

Eine gute Anforderungserhebung und Qualitätssicherung, welche Fehler früh findet & minimiert, ist dementsprechend essenziell. Dementsprechend müssen wir während unserer Schritte die Verifikation & Validierung durchführen, um dies mit Qualitätsmaßen vernünftig zu beurteilen.

8) Produkt / Deployment

Wenn unser Produkt fertig ist, ist es bereit zur Auslieferung an den Kunden. Da jedoch oftmals ein Produkt nicht nur ausgeliefert, sondern auch beim Kunden integriert wird, muss es auch gewartet & geupdated werden.

Genau für solche Cases bennötigen wir DevOps, welches Development und Operations verbindet. Wichtig hierfür sind kontinuierliche Integration & Deployment, um die Releases zu automatisieren, aber auch das Build-Management, Dependency-Management und Projektmanagement.

Schluss

Im Nachhinein gibt es noch eine abschließende Analyse über das Programm sowie evt. Erweiterungen, welche in weiteren Iterationen implementiert werden können.

Links & externe Tools

Um Diagramme zu erstellen, benutze ich primär draw.io, da Ich gute Erfahrungen mit dem Tool gemacht habe.

1) Anforderungsanalyse

Im ersten Schritt führen wir die Anforderungsanalyse durch. Allerdings müssen wir auch eine Sache nochmal besprechen:

Annahmen

Da die bereits vorgegebenen Anforderungen relativ wenig sind und viel Spielraum lassen, werden wir ein paar Annahmen machen müssen, um evt. Unsicherheiten zu behandeln. Wir dokumentieren hier Designannahmen, die wir über das Programm erheben.

- Das Programm läuft lokal auf einem Rechner.
- Spieler "Kreuz" & "Kreis" bedienen abwechselnd dasselbe Interface.
- Das Programm besitzt keine Netzwerkfunktion.
- Die Spielfeldgröße ist fest begrenzt auf 4x4.

Nun schauen wir uns die anderen Kernpunkte etwas genauer an:

A) Problemverständnis

Das gewünschte Programm ist eine Art "Anti-Tic-Tac-Toe". Es soll von zwei Spielern spielbar sein und die vorgegebenen Regeln einhalten. Da das Spielfeld ein 4x4 Feld ist, aber ein Verlust bei drei aneinander liegenden bereits gilt, müssen wir den Algorithmus später herunterbrechen.

B) funktionale Anforderungen

Hier unterteilen wir nochmal in "Muss-" und "Kann-"Anforderungen, um klarer eine Struktur zu bekommen und eine Art der Priorisierung vorzunehmen.

(Editors Note: Auch hier könnten wir weitere Unterscheidung in z.B. Spielregeln, Systemverhalten, etc. machen; für so ein kleines Programm jedoch erachte ich dies als nicht sinnvoll.)

Muss-Anforderungen:

- Das Programm soll ein Spielfeld in der Größe 4x4 besitzen.
- Das Spiel wird von zwei Spielern gespielt: "Kreuz" und "Kreis".
- Spieler setzen abwechselnd ihr Symbol in ein freies Feld.
- Ein Spieler verliert, sobald er drei seiner Symbole in einer Zeile, Spalte oder Diagonalen setzen muss.
- Das Spiel endet unentschieden, wenn alle Felder gesetzt sind, ohne dass ein Spieler verloren hat.

- Das Programm ermöglicht es zwei Spielern, gegeneinander zu spielen, und achtet dabei auf die Einhaltung dieser Regeln.

Kann-Anforderungen:

- Das Spiel zeigt an, welcher Spieler aktuell am Zug ist
- Das Programm verfügt über ein Menü, bei welchem man das Programm verlassen kann oder ein Spiel starten kann (beinhaltet evt. Namensvergabe)

C) nicht-funktionale Anforderungen

Da auch hier relativ wenig gegeben ist, treffen wir ein paar Annahmen:

- Das Spiel überprüft automatisch die Regeln
- ungültige Züge werden verhindert
- Das Spiel zeigt den aktuellen Zustand des Spielfeldes an (Konsistenz mit Data-Schicht)
- Das Spiel informiert über Sieg / Niederlage / Unentschieden
- der Code ist sauber strukturiert
- die Dokumentation ist vollständig und verständlich (mind. 90 % der Klassen & Funktionen)

D) Randbedingungen

Aufgrund der niedrigen Komplexität der Aufgabe, werde ich die Aufgabe mit Python bearbeiten. Da ich jedoch nicht wirklich Erfahrung mit GUI-Frameworks in Python habe, habe ich ChatGPT mir eine kleine Übersicht erstellen lassen, um einen Überblick zu bekommen.

Da ich wie bereits erwähnt keine Erfahrungen mit Python GUI-Frameworks habe und die Aufgabe recht simpel ist, werde ich bei der Python nativen GUI "Tkinter" bleiben, da dies ausreicht meine erhobenen Anforderungen zu implementieren.

2) Problemzerlegung / Algorithmen

Bevor wir zur Modellierung unseres Programmes kommen, müssen wir erst einmal etwas über das Herzstück des Programmes nachdenken: den Algorithmus zur Bestimmung von 3 nebeneinander stehenden Zeichen.

Dafür müssen wir uns erst einmal konzeptionell in die Problematik eindenken.

- Gegeben ist ein 4x4 Feld, auf welchem abwechselnd Kreuze und Kreise gesetzt werden.
- Verloren hat, wer als Erstes drei nebeneinander stehende Zeichen hat, horizontal, vertikal oder diagonal.

Berechtigterweise stellt sich nun ein Problem heraus: während der Case für einen Verlust auf 3x3 basiert, ist das Spielfeld 4x4 groß. Was tun wir also, damit wir vernünftig dies berechnen können?

Unser Algorithmus soll folgendes machen:

=> "Finde auf dem 4x4 Spielfeld jede Folge von 3 direkt aufeinanderfolgenden Feldern welche horizontal, vertikal oder diagonal sein können. Dazu definieren wir uns gültige Startblöcke, welche für die Überprüfung infrage kommen, um mit wenig Problemen diese Folgen zu finden."

Um jetzt die einzelnen Fälle abzudecken und auf Korrektheit zu prüfen, brauchen wir ein mathematisches Konstrukt, auf dem wir unseren Algorithmus basieren können.

Mathematische Basis:

Konzeptionell definieren wir jetzt gültige "Startpositionen" in unserem Feld, in welchen es Sinn macht, zu überprüfen, ob wir 3 aufeinanderfolgende Felder haben. Wenn wir zeigen können, dass wir alle Dreierblöcke auf alle gültigen Startpositionen abbilden können und ebenfalls andersherum, dass wir aus allen Startpositionen alle möglichen Dreierblöcke erreichen können insgesamt, so können wir garantieren, dass wir mit dieser Methode alle Fälle abdecken. Der Algorithmus wäre dann mathematisch vollständig und korrekt.

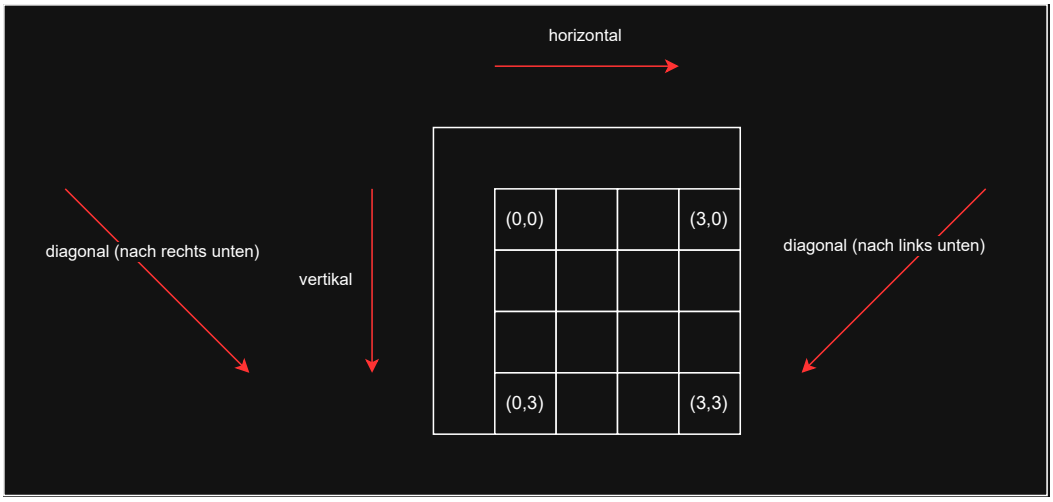
Definieren wir also Folgendes:

- Das Feld besteht aus Koordinaten, welche das Feld aufspannen
- Die Feldkoordinaten bestehen aus dem kartesischen Produkt von X & Y, wobei X = {0, 1, 2, 3} & Y = {0, 1, 2, 3}

	x = 0	x = 1	x = 2	x = 3
y = 0	(0,0)	...	...	(3,0)
y = 1	...	...	...	...
y = 2	...	...	...	...
y = 3	(0,3)	...	...	(3,3)

- Sei |X| = 4 als Breite definiert; |Y| = 4 als Höhe definiert
- Wir betrachten die Richtungen horizontal, vertikal, diagonal (nach rechts unten), diagonal (nach links unten)

- horizontal verläuft von links nach rechts, vertikal verläuft von oben nach unten; diagonal (nach rechts unten) verläuft von oben links nach unten rechts & diagonal (nach links unten) verläuft von oben rechts nach unten links



- Um unsere Fälle zu betrachten, müssen wir uns nun alle möglichen Startpositionen betrachten. Ein "Dreierblock" kann also nur da starten, wo noch zwei weitere Felder in der Richtung existieren. Formal heißt das für alle Fälle:

=> Menge horizontaler Startblöcke:

$$S_h : \{(x,y) \in X \times Y \mid (x,y),(x+1,y),(x+2,y) \in X \times Y\} = \{(0,0),(1,0),(0,1),(1,1),(0,2),(1,2),(0,3),(1,3)\}$$

=> Menge vertikaler Startblöcke:

$$S_v : \{(x,y) \in X \times Y \mid (x,y),(x,y+1),(x,y+2) \in X \times Y\} = \{(0,0),(0,1),(1,0),(1,1),(2,0),(2,1),(3,0),(3,1)\}$$

=> Menge diagonaler (nach rechts unten) Startblöcke:

$$S_{dru} : \{(x,y) \in X \times Y \mid (x,y),(x+1,y+1),(x+2,y+2) \in X \times Y\} = \{(0,0),(1,0),(0,1),(1,1)\}$$

=>Menge diagonaler (nach links unten) Startblöcke:

$$S_{dlu} : \{(x,y) \in X \times Y \mid (x,y),(x-1,y+1),(x-2,y+2) \in X \times Y\} = \{(2,0),(3,0),(2,1),(3,1)\}$$

(Editors note: Für eine zukünftige Änderung für den NxN-Fall müssten wir die Definition passend umbauen. Den Vollständigkeitsbeweis, dass für alle möglichen N die Startposition nur dann existieren kann, wenn zwei nachfolgende Felder existieren, könnte z.B.über vollständige Induktion bewiesen werden.

Ebenfalls wäre es theoretisch möglich, einen Richtungsvektor einzubauen, mit welchem wir dann einfach später mit einer Formel die jeweiligen möglichen Richtungen gehen könnten.)

Um nun die Korrektheit zu folgern, müssen wir zeigen, dass jede mögliche Dreierfolge genau so eine Startposition besitzt. Nur so ist unser Algorithmus auch vollständig. Da unser Feld recht klein ist, stelle ich jede mögliche Dreierfolge einem Startblock über. Wenn alle zugeteilt werden können, ist unsere Regel auch korrekt und wir können diese implementieren als Fallunterscheidungen.

(Editors note: Für den NxN reicht diese Definition / der Beweis natürlich nicht aus. Am besten sollten wir hierfür dann eine Funktion definieren für jeden Fall (horizontal, vertikal, etc.), welche von der Startposition auf das Tripel abbildet. Dann müssten wir noch beweisen, dass die Menge aller Bildmengen von unseren definierten Funktionen und die Menge aller möglichen Dreierfolgen beide gleich sind.)

Beweis:

=> horizontale Dreierfolgen

((0,0), (1,0), (2,0)) -> (0,0)
((1,0), (2,0), (3,0)) -> (1,0)
((0,1), (1,1), (2,1)) -> (0,1)
((1,1), (2,1), (3,1)) -> (1,1)
((0,2), (1,2), (2,2)) -> (0,2)
((1,2), (2,2), (3,2)) -> (1,2)
((0,3), (1,3), (2,3)) -> (0,3)
((1,3), (2,3), (3,3)) -> (1,3)

=> vertikale Dreierfolgen

((0,0), (0,1), (0,2)) -> (0,0)
((0,1), (0,2), (0,3)) -> (0,1)
((1,0), (1,1), (1,2)) -> (1,0)
((1,1), (1,2), (1,3)) -> (1,1)
((2,0), (2,1), (2,2)) -> (2,0)
((2,1), (2,2), (2,3)) -> (2,1)
((3,0), (3,1), (3,2)) -> (3,0)
((3,1), (3,2), (3,3)) -> (3,1)

=> Diagonale Dreierfolgen

((0,0), (1,1), (2,2)) -> (0,0)
((1,0), (2,1), (3,2)) -> (1,0)
((0,1), (1,2), (2,3)) -> (0,1)
((1,1), (2,2), (3,3)) -> (1,1)

((2,0), (1,1), (0,2)) -> (2,0)
((3,0), (2,1), (1,2)) -> (3,0)
((2,1), (1,2), (0,3)) -> (2,1)
((3,1), (2,2), (1,3)) -> (3,1)

==> Damit ist der Algorithmus mathematisch vollständig.

Später werden wir diese Fallunterscheidung nutzen, um gleiche Nachbarn zu analysieren, damit wir evt. Ketten oder 'hits' finden können.

Der Algorithmus logisch beschrieben:

Der später implementierte Algorithmus wird dann so funktionieren:

Der Rückgabewert der Funktion ist ein Boolean, true für "Block gefunden" und false für "kein Block vorhanden".

Als Basis für unsere Datenstruktur haben wir ein Grid [x][y] in welchem wir unsere Koordinaten stehen haben. Dieses Grid wird so implementiert, dass es 3 mögliche Einträge / Zustände für ein einzelnes Feld in diesem geben kann:

1. "0" = Das Feld ist leer und kein Zeichen ist gesetzt
2. "1" = Spieler A hat das zum Spieler zugehörige Zeichen A gesetzt (Kreuz oder Kreis)
3. "2" = Spieler B hat das zum Spieler gehörige Zeichen B gesetzt (das jeweils andere, was Spieler A nicht besitzt)

Außerdem definieren wir eine Hilfsfunktion, welche am Anfang des Spiels unsere Startpositionen berechnet und in einem Array speichert. Zwar ist dies für den 4x4-Fall nicht zwingend notwendig (sie könnten auch einfach hart codiert werden), aber interessant für eine mögliche NxN-Erweiterung, welche dann dynamisch diese berechnen kann.

Warum ist gerade so eine Hilfsfunktion nun interessant oder gar notwendig?

Wir haben bereits bewiesen, dass unser Konstrukt aus Startpositionen mathematisch vollständig und korrekt ist. Diese Eigenschaft machen wir uns nun zunutze, da wir wissen, dass ein Dreierblock nicht ohne Startposition existieren kann. D.h. unser Algorithmus muss erst dann auf einen möglichen Verlust prüfen, wenn ein Zeichen auf eine Startposition gesetzt wurde.

Dies ist effizienter, da der Algorithmus nicht auf allen möglichen Koordinaten ausgeführt wird. Ebenfalls erleichtert dies uns, in weniger "Out-of-Bounds"-Fälle potentiell zu kommen, da wir wissen, dass die Fälle horizontal & vertikal nicht über die Grid-Koordinaten hinausgehen (Diagonal ist hier ein Spezialfall dann).

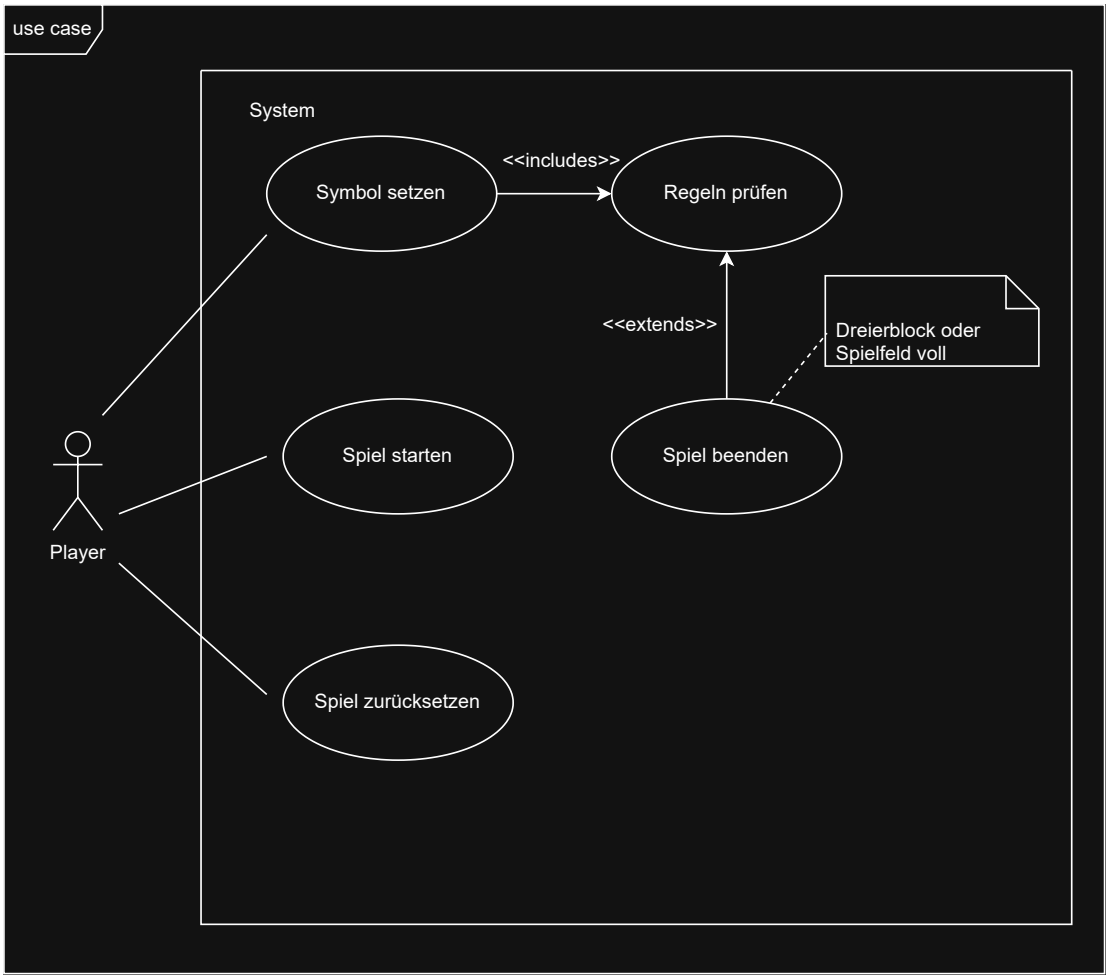
Der Algorithmus wird später nach Abschluss des Zuges des gerade aktiven Spielers ausgeführt (Siehe Sequenzdiagramm zu Spielzug). Der Algorithmus berechnet dann Folgendes:

1. Sammle alle Koordinaten von Einträgen im Grid des aktiven Spielers als Array Ka
2. Filter die Einträge aus Ka auf Startpositionen, indem wir den Schnitt der beiden Mengen bestimmen
3. Wenn der Schnitt leer ist, hören wir mit dem Algorithmus auf (false) => weiter im Sequenzdiagramm
4. Ist der Schnitt nicht leer, so müssen wir uns die Nachbarfelder der Koordinate angucken und schauen, ob dort ebenfalls ein Zeichen gesetzt ist (wir geben die Koordinate dann an unsere Hilfsfunktionen, welche überprüfen, ob ein horizontaler, vertikaler oder diagonaler Block besteht)
5. Ist die Analyse unserer Hilfsfunktionen negativ, so hören wir mit dem Algorithmus auf (false) => weiter im Sequenzdiagramm
6. Ist die Analyse unserer Hilfsfunktion positiv, so geben wir diese Info weiter (true) => weiter im Sequenzdiagramm

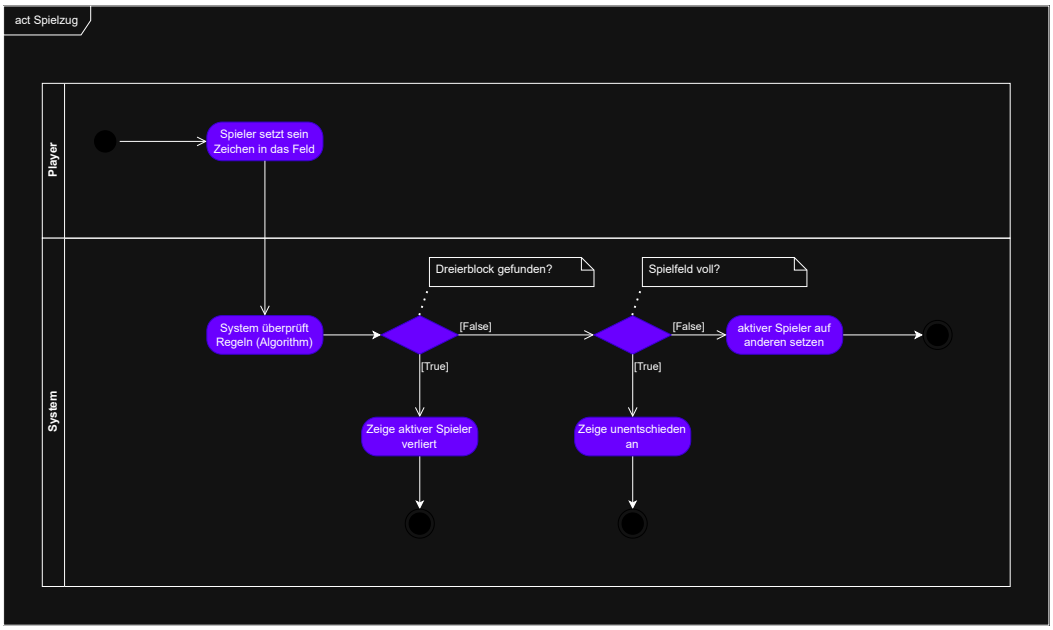
3) Modeling / Verständnis

Auf Basis unserer Anforderungsanalyse können wir unsere Anforderungen modellieren, um ein technisches Verständnis der Abläufe des Programmes zu bekommen. Dazu werden wir folgende Diagramme benutzen:

- Use-Case-Diagramm: Interaktion zwischen Spieler & System



- Aktivitätsdiagramm: Ablauf eines Spielzuges



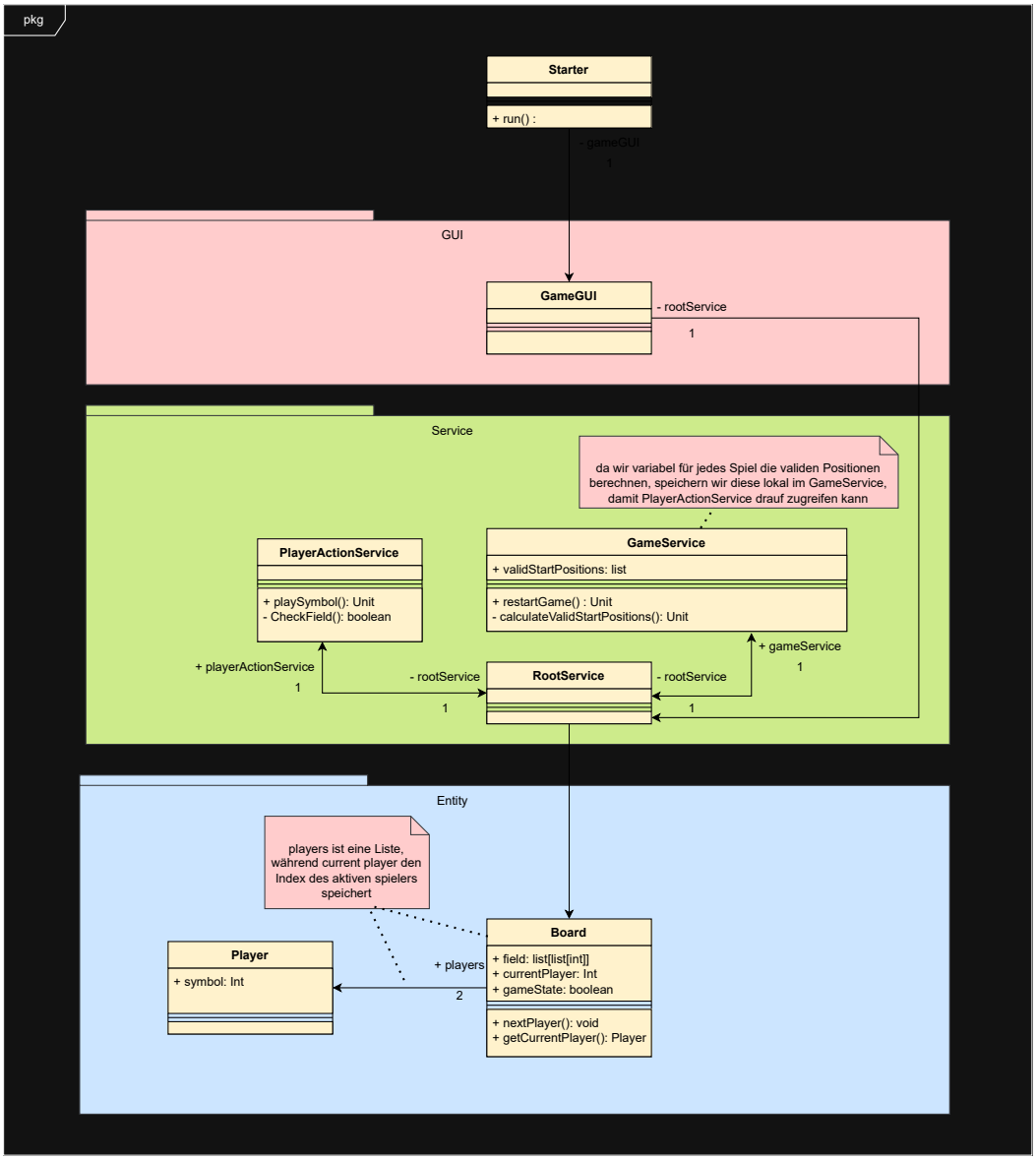
4) Architektur & Strukturdesign

Nachdem wir die Abläufe der Software modelliert haben, müssen wir uns nun Gedanken um die Architektur machen. Da das Programm recht simpel ist, reicht es aus, ein einfaches Klassendiagramm zu modellieren.

Allerdings treffe ich hier eine Entscheidung, um die Aufgabe etwas einfacher zu machen:

Anstatt eine Startseite zu implementieren, über welche man ein Spiel starten kann, werde ich einfach nur das Gamefield direkt implementieren.
Dies hat den Grund, dass die Null-Safety in Python nicht unbedingt gegeben ist.
Außerdem möchte ich mich auf ein funktionsfähiges Spiel fokussieren.
Dementsprechend entfällt der oben modellierte Use-Case "Spiel starten" (Use-Case-Diagramm).

- Klassendiagramm:



Nach dieser Iteration unserer Anforderungsanalyse & Modellierung haben wir nun einen ersten State, welchen wir implementieren & testen können.

5 + 6) Implementierung & Testing

Da es schwierig ist, Code, welcher in verschiedenen Klassen liegt, in so einem Dokument festzulegen, habe ich meine Implementierung auf meinem GitHub Account hinterlegt.

https://github.com/Thopia03/3VerliertGame

Ebenfalls beinhaltet es dieses Dokument, als auch die einzelnen Diagramme.

1) Iteration - Entity-Schicht

Nach erster Iteration der Implementierung habe ich ein vernünftiges Testing der Entity-Schicht vorgenommen. Dies hat den Grund, dass die Objekte der Entity-Schicht essenziell sind, damit der Code auch nicht auf falschen Daten arbeitet.

Die Entity-Schicht ist nun vollständig implementiert, dokumeniert und hat bis jetzt keine Probleme oder Bugs hervorgehoben. Dementsprechend gehen wir weiter in die nächste Iteration, die Implementierung der Service-Schicht.

2) Iteration - Service-Schicht

Nach erster Implementierung & Testing habe ich bemerkt, dass es schlauer ist, für die validStartingPositions ein Set zu nutzen, da wir dort später einfacher die Mengenoperationen machen können für unsere Koordinaten. Dementsprechend habe ich das nachträglich im Klassendiagramm geändert. Sonst sind die Tests des GameService positiv und ohne Probleme abgelaufen und ich kann weiter am playerActionService arbeiten.

Allerdings ist mir aufgefallen dass, wenn man mit Python zweidimensionale Koordinaten abbildet, man Listen von Listen hat. Dementsprechend muss man bei der Art 'field[][]' der Referenzierung die Koordinaten anders referenzieren, da wir mit dieser Art erst die vertikalen und dann die horizontalen Koordinaten referenzieren, sprich 'field[y][x]'.

Nach der nächsten Phase funktionieren unsere Hilfsfunktionen jetzt ohne Probleme. Nun müssen wir checken, ob unsere Hauptfunktionen funktionieren, damit wir nach dem Testing unsere GUI implementieren können.

Auch die Tests der Hauptfunktionen sind positiv verlaufen, weswegen wir uns jetzt dem letzten Teil der Implementierung widmen können, der GUI unserer Applikation. Wie bereits in der Anforderungsanalyse erwähnt, nutzen wir Tkinter um diese zu bauen. Dabei legen wir Wert auf Funktionalität zunächst, bevor wir ein schönes Produkt später liefern.

In Anbetracht der restlichen Zeit, die wir besitzen, wird dies auch so bleiben. Dementsprechend gehen wir jetzt in die letzte Phase der Implementierung.

3) Iteration - GUI-Schicht und Starter-Klasse

Nach dritter Implementierung ist die GUI nun auch vollständig einsetzbar. Klasse! Teil 7 haben wir aufgrund der geringen Größe des Projektes bereits in der Implementierung mit dem Testing abgefangen und alle Anforderungen, die gegeben wurden, sind erfolgreich umgesetzt worden.

8) Deployment

Da wir mit der Implementierung in der ersten Phase durch sind, ist das letzte Unterfangen, das wir haben, unser Deployment der Software. Da ich für diese Aufgabe kein CI/CD machen werde, ist der Fokus hier auf das erfolgreiche Replizieren der Datei der Hauptfokus. Dafür werde ich eine README noch nacharbeiten, welche vernünftig den Code replizieren lässt.

Schlussteil

Zu dem Zeitpunkt, wo ich dies hier schreibe, bin ich gerade dabei, die Anforderungsanalyse abzuschließen. Allerdings sind mir ein paar Ideen gekommen, wie man das Programm in weiteren Iterationen erweitern könnte, weswegen ich diese schon einmal hier dokumentiere.

- Einzelspielermodus gegen einen Bot (Random, Monte-Carlo, Minimax)
- Anzeigen einer Spielhistorie
- Erweiterung des Feldes auf NxN-Größe (weitere Erklärung hierzu gibt es in den Editors-Notes zur Problemzerlegung / Algorithmen)

Weitere Anforderungen oder eher Features, die nach Abschluss der Implementierungsphase aufgekommen sind, könnten sein:

- Customizable Theme
- Menu-Scene, über welche man Namen festlegen kann oder auch Zeichen, mit welchen man custom spielen kann

Außerdem möchte ich noch eine abschließende Analyse abgeben. Das Programm läuft bis jetzt ganz gut, allerdings könnten einige Sachen in der Modellierung verbessert werden. Das Attribut "validStartPositions" in der GameService-Klasse könnte in den PlayerActionService verlegt werden, um die Kopplung zwischen den Services zu verringern.

Laufzeit

Ebenfalls stellt sich die Frage, ob der Algorithmus mit den 'validen Startpositionen' wirklich für einen NxN-Fall effizienter wäre.

Wir schauen uns den Algorithmus einmal an:

"1) Collect all coordinates of the symbols placed by the current player.":

Im Worst-Case bauen wir uns ein Feld mit der Größe $O\left(\frac{n^2}{2}\right) = O(n^2)$
Ebenfalls gehen wir durch das ganze Feld, dementsprechend haben wir hier auf jeden Fall eine Laufzeit von $O(n^2)$

"2) Now we filter the list by intersecting the coordinates with the valid start positions.":

Wir definieren

- A = playerCoordinates
- B = validStartPositions

Die Komplexität unseres Schnittes ist $O(\min(|A|, |B|))$ Für ein NxN-Feld gilt, dass validStartPositions $\approx O(n^2)$, da wir nur konstant 25 % einsparen und die Anzahl der möglichen Startpunkte quadratisch wächst.
Dementsprechend gilt für Worst-Case an Speicherplatz:
$O(\min(|A|, |B|)) = O(\min(O(n^2), O(n^2))) = O(n^2)$ (auch für Laufzeit, da wir quadratisch große Mengen miteinander vergleichen).

3 brauchen wir nicht zu analysieren, da es eine einfache Überprüfung ist.

"4) Now that there are valid start positions, we need to check their neighbours for a hit."

Wir haben hier vier Methoden, welche die Überprüfung durchführen auf unseren playerCoordinates:

- checkHorizontal
- checkVertical
- checkDiagonalRightDown
- CheckDiagonalLeftDown

Worst-case haben wir hier $|playerCoodinates| = O(n^2)$ viele Koordinaten die wir überprüfen in der Laufzeit. Da die Funktionen selber nur konstant viele Feldzugriffe (zwei) machen, bleibt die Laufzeit $O(n^2)$. Auch die Anzahl der Überprüfungen (vier) bleibt gleich, dementsprechend bleibt unsere Laufzeit hier $O(n^2)$

Da die anderen beiden Cases wie 3 einfache Überprüfungen für "checkField" sind, können wir die Gesamtlaufzeit berechnen.

Gesamtlaufzeit

Wenn wir alle Cases zusammenführen, berechnet sich unsere Laufzeit als:
$T(n) = O(n^2) + O(n^2) + O(n^2) = O(n^2)$

Unser Speicherverbrauch bezieht sich durch die beiden Strukturen "playerCoordinates" und "ValidStartPositions" auf $S(n) = O(n^2) + O(n^2) = O(n^2)$

Für unseren 4x4-Fall ist dies unproblematisch, da wir konstant hier bleiben.

Ist unser Algorithmus nun unnötig kompliziert?

Meiner Meinung nach ist der Algorithmus ein interessanter Approach bezüglich des Problems, er spart jedoch nichts an Komplexität ein. Alternativ könnte man auch einfach einen einfachen Algorithmus beschreiben, welcher nur vom zuletzt plazierten Feld aus

prüft. Dies wäre für einen NxN-Fall wahrscheinlich einfacher zu lösen und weniger komplex, allerdings würde auch hierzu eine vernünftige Analyse fällig.

Abschließende Worte

Wie bereits ganz oben erwähnt, befindet sich der Source-Code auf meinem GitHub-Account unter https://github.com/Thopia03/3VerliertGame

Bei Fragen oder Feedback bin ich gerne jederzeit erreichbar.

Danke für die interessante Aufgabe und ich freue mich auf eine positive Rückmeldung!

- Nico Rautert (Thopia)

Sources & Links:

https://www.drawio.com
https://docs.python.org/3/library/tkinter.html